

Java Backend Architecture

Interview Series

Chapter 16.1

Docker Architecture

50+ Interview Q&A

Code Examples

Architecture Diagrams

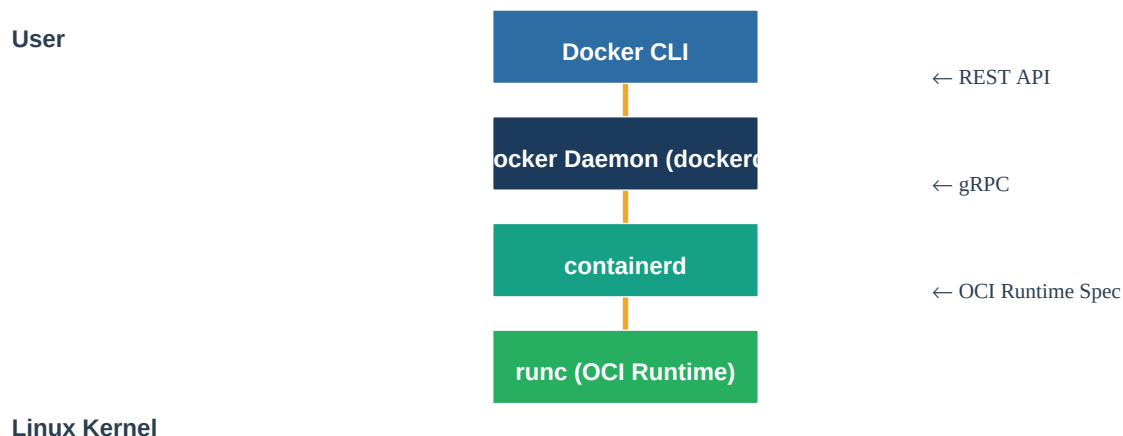
Advanced Topics

Chapter 16.1 – Docker Architecture

Docker Architecture Overview

Docker is a platform for building, shipping, and running containerized applications. Understanding its internal architecture is essential for optimizing builds, debugging issues, and making informed decisions about container orchestration.

Docker Component Stack



Core Components

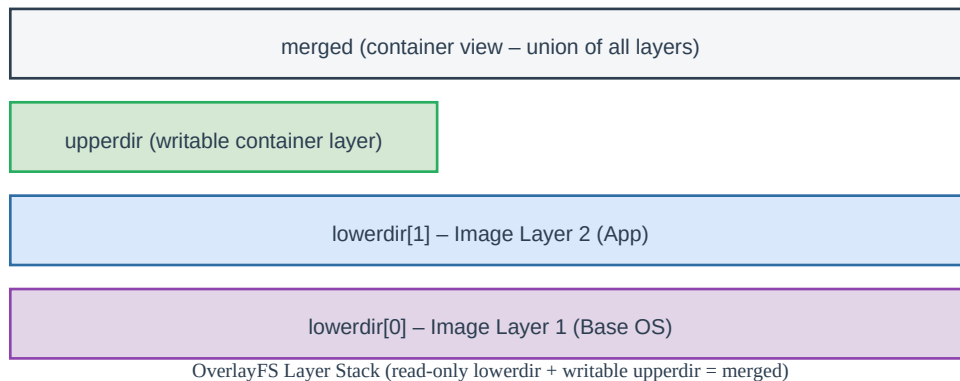
Docker CLI	Command-line tool that sends REST API requests to Docker daemon. Translates user commands (docker run, build, push) into API calls.
dockerd (daemon)	The Docker daemon process. Listens on Unix socket /var/run/docker.sock. Manages images, containers, networks, volumes via REST API.
containerd	High-level container runtime. Manages container lifecycle: create, start, stop, delete. Pulls and stores images. Communicates with runc via containerd-shim.
runc	Low-level OCI-compliant runtime. Creates containers by setting up Linux namespaces and cgroups. Spawns container process and exits.
containerd-shim	Per-container process that keeps running after runc exits. Allows containerd to restart without affecting running containers.

OCI Specifications

The Open Container Initiative (OCI) defines two key specifications that ensure interoperability across container runtimes and registries:

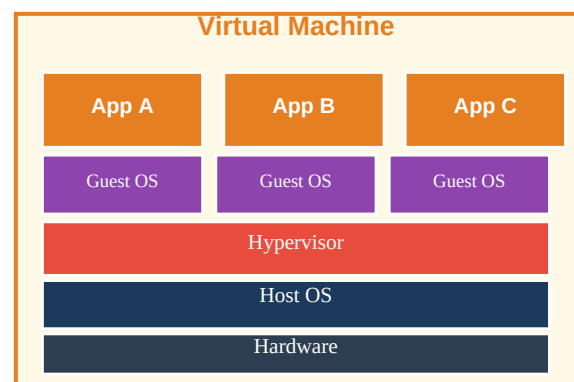
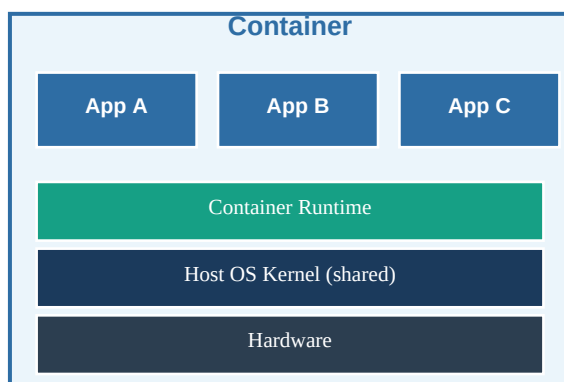
- Image Spec: Defines image layers (compressed tarballs), config (env/cmd/entrypoint), and manifest (list of layers + config digest)
- Runtime Spec: Defines container configuration (rootfs, process, mounts, namespaces, cgroups) via config.json
- Distribution Spec: Defines how images are pushed/pulled from registries using HTTP API

OverlayFS – Union Filesystem



Docker uses OverlayFS as its default storage driver. Each image layer is a lowerdir (read-only). The running container adds an upperdir (writable). The merged view combines all layers using copy-on-write semantics — only changed files are written to upperdir.

Container vs Virtual Machine



Aspect	Container	Virtual Machine
OS Kernel	Shares host kernel	Full guest OS per VM
Startup time	Milliseconds	Seconds to minutes
Image size	MB (tens to hundreds)	GB
Isolation	Process-level (namespaces)	Hardware-level (hypervisor)
Overhead	Near-zero	CPU/RAM for each guest OS
Portability	High (OCI standard)	Moderate (hypervisor-specific)

Linux Namespaces

Docker uses six Linux namespaces to provide process isolation within containers:

PID namespace	Container gets its own PID space. Container PID 1 = first process (e.g. Java app). Host sees different PIDs.
NET namespace	Container gets its own network stack: interfaces, routing tables, iptables rules, sockets.

MNT namespace	Container gets its own filesystem mount points. Enables container root filesystem isolation.
UTS namespace	Container can have its own hostname and domain name, independent of host.
IPC namespace	Container gets isolated inter-process communication resources (message queues, semaphores, shared memory).
USER namespace	Maps container user IDs to different host user IDs. Enables rootless containers.

cgroups (Control Groups)

Linux cgroups v2 limit and account for resource usage by container processes:

Resource	Docker Flag	Description
CPU	--cpus 1.5	Limit to 1.5 CPU cores using CPU bandwidth controller
CPU set	--cpuset-cpus 0,1	Pin container to specific CPU cores
Memory	--memory 512m	Hard memory limit; OOMKiller triggers if exceeded
Memory swap	--memory-swap 1g	Total memory+swap limit
Block I/O	--blkio-weight 500	Proportional block I/O access weight (100-1000)
PIDs	--pids-limit 100	Prevent fork bombs by limiting number of processes

Docker Rootless Mode

Rootless Docker runs the daemon and containers without root privileges. The daemon runs as a normal user using user namespaces. This significantly reduces attack surface — a container escape does not grant root access on the host. Trade-offs include limited network functionality (no host networking, limited port mapping below 1024).

```
# Install rootless Docker dockerd-rootless-setuptool.sh install # Run as non-root user export
DOCKER_HOST=unix:///run/user/1000/docker.sock docker run hello-world
```

Interview Q&A; – Docker Architecture

Q1. What is the role of the Docker daemon (dockerd)?

The Docker daemon is the persistent background service that manages all Docker objects. It listens on a Unix socket (`/var/run/docker.sock`) for REST API requests from the Docker CLI or other clients. It handles image management (pull/push/build), container lifecycle (create/start/stop/kill), networking (create/manage networks), and volume management. It delegates actual container execution to containerd.

Q2. What is containerd and how does it differ from dockerd?

containerd is a high-level container runtime that manages the complete container lifecycle: image pull, storage, container creation, execution, supervision, and deletion. dockerd is the Docker daemon that provides the Docker API and high-level features (build, Swarm). dockerd uses containerd as a backend runtime. containerd can also be used independently (e.g. by Kubernetes via CRI).

Q3. What is runc and what does it do?

runc is a lightweight CLI tool that implements the OCI Runtime Specification. It reads a container's `config.json` (generated by containerd), sets up Linux namespaces and cgroups, creates the rootfs mount, and spawns the container process. runc exits after starting the container — the container-shim process then keeps the container running and reports status back to containerd.

Q4. Explain the OCI Image Specification.

The OCI Image Spec defines how container images are structured: (1) Image manifest: JSON document listing the config blob digest and ordered list of layer digests. (2) Image config: JSON blob containing environment variables, CMD, ENTRYPOINT, exposed ports, and the `diff_ids` for each layer. (3) Layers: compressed tar archives (gzip or zstd) representing filesystem changes. Images are content-addressable — identified by SHA256 digest of the manifest.

Q5. How does OverlayFS work in Docker?

OverlayFS is Docker's default storage driver (overlay2). It stacks read-only image layers as `lowerdirs` and adds a single writable `upperdir` for the container. The merged directory presents a unified view. When a container modifies a file from a lower layer, OverlayFS copies the file up to `upperdir` first (copy-on-write). Deleted files create whiteout files in `upperdir`. This allows fast container creation without copying entire image layers.

Q6. What are the 6 Linux namespaces Docker uses?

Docker uses: PID (process isolation, container gets PID 1), NET (network stack isolation), MNT (filesystem mount isolation), UTS (hostname isolation), IPC (inter-process communication isolation), and USER (UID/GID mapping for rootless). Each namespace creates an isolated view of that system resource for processes in the container, preventing interference with the host or other containers.

Q7. What are Linux cgroups and how does Docker use them?

Control groups (cgroups) are a Linux kernel feature that limits, accounts for, and isolates resource usage of process groups. Docker uses cgroups v2 to enforce: CPU limits (`--cpus`), memory limits (`--memory`, with OOMKiller enforcement), block I/O limits (`--blkio-weight`), and PID limits (`--pids-limit`). When a container exceeds its memory limit, the Linux OOM Killer terminates a process in the container, resulting in an OOMKilled exit state.

Q8. What is the difference between a container and a virtual machine?

Containers share the host OS kernel and use namespaces/cgroups for isolation — they are processes with restricted views. VMs run a complete guest OS on top of a hypervisor with full hardware emulation.

Containers start in milliseconds, use MB of disk, and have near-zero overhead. VMs start in seconds/minutes, use GB, but offer stronger isolation (hypervisor boundary). Containers are better for microservices density; VMs for multi-tenant strong isolation.

Q9. What is containerd-shim and why is it needed?

The containerd-shim is a per-container process that sits between containerd and the container's init process. It serves three purposes: (1) Allows containerd to restart or upgrade without killing running containers. (2) Reaps zombie processes (acts as subreaper). (3) Reports container exit status back to containerd via its API. Without the shim, restarting containerd would orphan all running containers.

Q10. What is Docker rootless mode and when should you use it?

Rootless Docker runs the entire Docker stack (daemon + containers) as a non-root user using user namespaces. A container escape vulnerability cannot give the attacker root access to the host. Use rootless mode in environments requiring strong security boundaries, CI/CD systems, or multi-user machines. Limitations include: no host networking, port numbers below 1024 require workarounds, and some storage drivers are unsupported.

Q11. What REST API does Docker use internally?

Docker uses a versioned REST API over a Unix socket (`/var/run/docker.sock`) by default, or TCP with TLS for remote access. The API version is in the URL path (e.g. `/v1.44/containers/json`). The Docker CLI is a REST client — every docker command translates to one or more API calls. Third-party tools like Portainer, VS Code Docker extension, and Kubernetes all use this API. You can interact directly with curl: `curl --unix-socket /var/run/docker.sock http://localhost/v1.44/info`

Q12. How does Docker handle image layer caching?

Each image layer is identified by its SHA256 content digest. Docker's layer store keeps layers on disk; if a pull detects a layer digest already present locally, it reuses it. During builds, each Dockerfile instruction that changes the filesystem creates a new layer. Build cache hits occur when the instruction and all parent layers match. The overlay2 driver stores layers in `/var/lib/docker/overlay2/` as directories with link, diff, lower, work, and merged subdirectories.

Q13. What is the difference between docker run and docker exec?

`docker run` creates and starts a NEW container from an image. `docker exec` runs a command in an ALREADY RUNNING container by creating a new process that joins the container's existing namespaces (PID, NET, MNT, etc.). `docker exec -it sh` is commonly used for interactive debugging — it gives you a shell inside the running container without starting a new container or interrupting the main process.

Q14. Explain the Docker REST API socket security implications.

The Docker socket `/var/run/docker.sock` grants full control over the Docker daemon — equivalent to root access on the host. Anyone with socket access can run privileged containers, mount host filesystem, and escape to the host. Never mount the Docker socket into containers unless absolutely necessary (e.g. CI/CD Docker-in-Docker). Use alternatives like Kaniko (rootless builds), Buildah, or Podman for container builds in CI/CD pipelines without socket access.

Q15. What storage drivers does Docker support besides overlay2?

Docker supports: overlay2 (default, recommended for all modern Linux), devicemapper (legacy, deprecated), btrfs (Btrfs filesystem required), zfs (ZFS filesystem required), vfs (no copy-on-write, for testing only — copies entire filesystem per container). overlay2 is the recommended driver for production — it uses the kernel's native OverlayFS, requires no additional filesystem, and has excellent performance. Always use overlay2 unless you have specific Btrfs/ZFS requirements.

Q16. How do you inspect the internal state of a running container?

Use `docker inspect` to get a JSON document with: Config (Env, Cmd, Entrypoint, Image, Labels), HostConfig (resource limits, mounts, network settings), NetworkSettings (IP addresses, ports, networks), State (Running, Pid, ExitCode, OOMKilled, Health), Mounts (source, destination, mode). You can use Go template syntax to extract specific fields: `docker inspect --format '{{.State.Pid}}'`

Q17. What is the Docker build context and why does it matter?

The build context is the directory (or URL) sent to the Docker daemon when running `docker build`. The entire context is tarred and sent to the daemon before building starts — even files not referenced in the Dockerfile. Large contexts slow down builds and waste bandwidth. Use `.dockerignore` to exclude `node_modules`, `.git`, `target/`, test data, and other unnecessary files. A lean build context (< 10 MB) dramatically speeds up builds, especially for remote daemons or CI/CD.

Q18. What is Docker Content Trust (DCT)?

Docker Content Trust uses Notary (based on The Update Framework) to sign and verify images. When `DOCKER_CONTENT_TRUST=1`, `docker pull` verifies the image signature before allowing the image to run. Publishers sign images with their private key; clients verify with the public key from the Notary server. DCT prevents pulling tampered or unsigned images. For modern use, Cosign + Sigstore is preferred as it integrates better with OCI registries and keyless signing.

Q19. Explain the difference between CMD and ENTRYPOINT in Docker.

ENTRYPOINT sets the executable that always runs (default: `/bin/sh -c`). CMD provides default arguments to ENTRYPOINT, or the command to run if ENTRYPOINT is not set. When both are set with exec form: ENTRYPOINT is PID 1, CMD args are passed to it. CMD can be overridden at docker run time; ENTRYPOINT requires `--entrypoint` flag to override. Best practice: use ENTRYPOINT for the main executable and CMD for default arguments, enabling easy parameter overriding.

Q20. What happens when a container's PID 1 exits?

When PID 1 (the init process) of a container exits, the container stops. All other processes in the container are killed with SIGKILL. The exit code of PID 1 becomes the container's exit code (0 = success, non-zero = error). If using CMD/shell form, the shell is PID 1 and your app is a child — the shell may not propagate SIGTERM correctly. Use exec form (ENTRYPOINT `["java", "-jar", "app.jar"]`) so your Java process is PID 1 and receives signals directly.

Q21. How does Docker networking work at the kernel level?

Docker creates a virtual Ethernet pair (veth) for each container. One end (eth0) is placed in the container's NET namespace; the other end is attached to the `docker0` bridge on the host. The bridge performs NAT using iptables MASQUERADE rules for outbound traffic. Port mapping adds DNAT rules in iptables PREROUTING and FORWARD chains. For user-defined networks, Docker's embedded DNS server (127.0.0.11) resolves container names using the network's built-in DNS.

Q22. What is the Docker Hub rate limit and how do you work around it?

Docker Hub limits unauthenticated pulls to 100 per 6 hours per IP, and authenticated free accounts to 200 per 6 hours. Workarounds: (1) Authenticate with `docker login` to get higher limits. (2) Use a pull-through registry cache (`docker pull` rate is shared across your org). (3) Mirror base images to a private registry (ECR, GCR, Harbor). (4) Use GitHub Container Registry (`ghcr.io`) or Quay.io as alternatives with more generous limits. CI/CD should always authenticate and use a mirror.

Q23. What is multi-platform Docker build and how does it work?

Multi-platform builds (`docker buildx build --platform linux/amd64,linux/arm64`) create images that run on multiple CPU architectures. BuildKit uses QEMU for cross-compilation or native build nodes. The result is an OCI image index (manifest list) with per-platform manifests. When you `docker pull`, Docker automatically selects the correct platform. This is essential for supporting both Intel/AMD servers and ARM-based instances (AWS Graviton, Apple Silicon).

Q24. Explain Docker's image garbage collection.

Unused images accumulate on disk. `docker image prune` removes dangling images (untagged layers no longer referenced). `docker image prune -a` removes all images not used by any container. `docker system prune` removes stopped containers, dangling images, unused networks, and build cache. Production systems should implement automated cleanup: Docker daemon's `--max-concurrent-downloads` and registries' lifecycle policies. `containerd` also has its own garbage collector for the image store.

Q25. What is the difference between `docker stop` and `docker kill`?

`docker stop` sends `SIGTERM` to PID 1, waits for a grace period (default 10 seconds), then sends `SIGKILL` if still running. This allows the application to perform graceful shutdown (flush buffers, close connections, complete in-flight requests). `docker kill` sends `SIGKILL` (or a specified signal) immediately without waiting — the process is terminated immediately. Always prefer `docker stop` for production containers. Spring Boot apps should handle `SIGTERM` to trigger graceful shutdown.

Q26. How does Docker handle DNS resolution?

Containers on default bridge get `/etc/resolv.conf` pointing to the host's DNS. Containers on user-defined networks get `/etc/resolv.conf` pointing to 127.0.0.11 (Docker's embedded DNS). The embedded DNS resolves container names within the same user-defined network, enabling service discovery by name (e.g. `db`, `redis`). The DNS search domain is set to the network name. External names are forwarded to the host's DNS resolver. `docker run --dns` overrides DNS server.

Q27. What are the key differences between `overlay2` and the deprecated `devicemapper`?

`overlay2` uses the kernel's OverlayFS and operates at the file level — only changed files are written to the container layer. It requires a Linux filesystem supporting `d_type` (`ext4`, `xfs` with `fmatch` enabled). `devicemapper` uses thin provisioning at the block level — it allocates thin block devices per container/image. `devicemapper` loop-back mode (default in old Docker) was extremely slow. `overlay2` has better performance, simpler setup, and is the only recommended driver for modern Linux distributions.

Q28. What is BuildKit and what improvements does it bring?

BuildKit is Docker's modern build engine (enabled by default since Docker 23). Improvements over legacy builder: (1) Parallel stage execution (multi-stage builds run stages concurrently when possible). (2) Cache mounts (`--mount=type=cache`) for build caches like Maven `.m2` or npm cache. (3) Secret mounts (`--mount=type=secret`) for build-time secrets that don't appear in layers. (4) SSH mounts for private repo access. (5) Lazy evaluation — only builds what's needed for the target stage. (6) Better progress output.

Q29. How does Docker implement port mapping?

`docker run -p 8080:80` creates iptables rules: PREROUTING DNAT rule redirects packets arriving at host port 8080 to the container's IP:80. A FORWARD rule allows forwarding between host network and `docker0` bridge. The `docker-proxy` process (optional) also listens on the host port for localhost traffic that bypasses iptables. Port mapping works by NAT — the container doesn't know it's being accessed on a different port than its own.

Q30. What is the `/var/run/docker.sock` and what are its permissions?

The Docker socket is a Unix domain socket at `/var/run/docker.sock` owned by root with group `docker` (usually 660 permissions). Users in the `docker` group can communicate with the daemon without `sudo` — equivalent to root access. In containers, mounting the socket enables the container to control Docker on the host

(Docker-in-Docker pattern). This is a major security risk — avoid it in production. Use dedicated CI/CD tools like Kaniko, Buildah, or Docker's rootless mode instead.

Q31. Explain Docker image digest vs tag.

A tag (e.g. `myimage:1.0`) is a mutable pointer — it can be updated to point to a different image over time. A digest (e.g. `myimage@sha256:abc123...`) is an immutable content-addressable reference to the exact image manifest. For production deployments, always use digests to ensure you're deploying exactly what was tested. Tags like `latest` are especially dangerous in production. CI/CD pipelines should pin deployment manifests to the image digest: `docker pull myimage:1.0@sha256:...`